Kiến trúc phần mềm

# Thiết kế cho bảo mật

ThS. Trần Mạnh Cường

# Đặt vấn đề

Hệ thống có 10 microservice. Mỗi service **tự** kiểm tra token, **tự** cấu hình HTTPS, **tự** chặn spam...

**Chỉ cần MỘT service quên kiểm tra?**

**CẢ HỆ THỐNG BỊ LỘ!**

→ Bảo mật phải được thiết kế vào kiến trúc, không để từng service tự lo.

# Nội dung

## 1. Bảo vệ điểm truy cập & dữ liệu

### Gatekeeper
Người gác cổng: kiểm tra, làm sạch request trước khi vào hệ thống

### Valet Key
Chìa khóa có hạn để client truy cập thẳng tài nguyên

### Sidecar
Tiến trình chạy kèm, gánh phần bảo mật cho ứng dụng

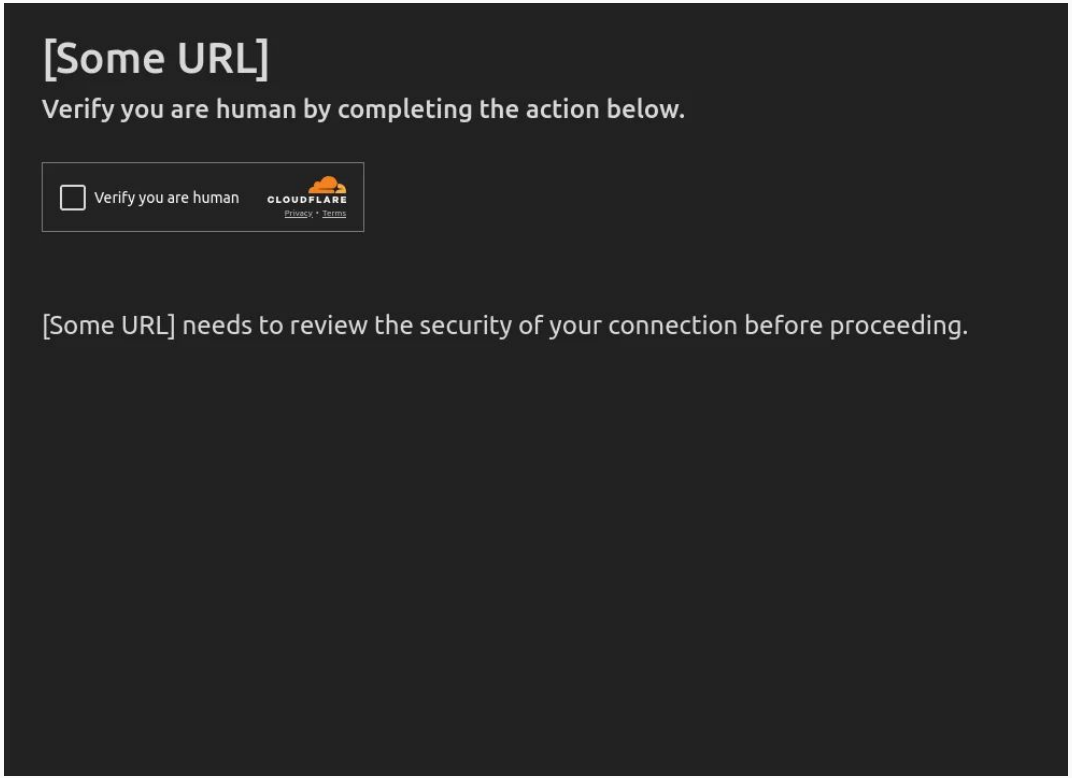
## 2. API Gateway: một cửa vào cho mọi request

### Routing
Một địa chỉ duy nhất, định tuyến tới nhiều service

### Aggregation
Gộp nhiều lượt gọi thành một

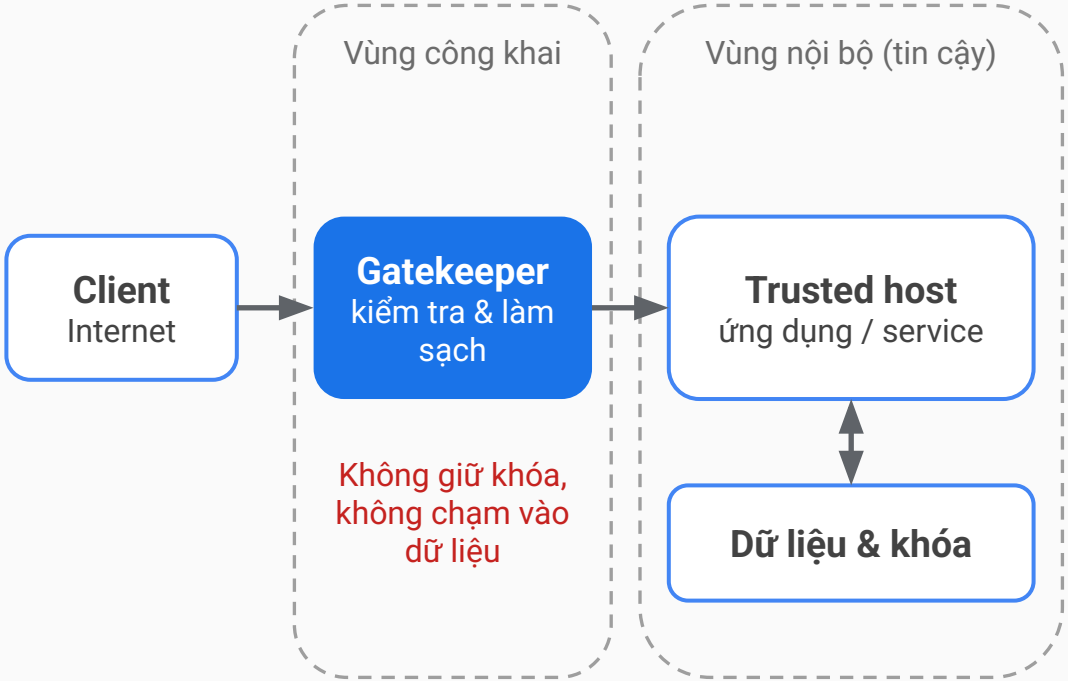
### Offloading
Gánh hộ TLS, xác thực, giới hạn tần suất...

# Gatekeeper


[Some URL]
Verify you are human by completing the action below.
Verify you are human
CLOUDFLARE
Privacy • Terms
[Some URL] needs to review the security of your connection before proceeding.

# Gatekeeper

Ý tưởng: đặt một “người gác cổng” giữa client và ứng dụng: kiểm tra, làm sạch request rồi mới chuyển vào trong.




## Ưu điểm

- Thu hẹp bề mặt tấn công: Internet chỉ thấy gatekeeper
- Gatekeeper bị chiếm vẫn chưa lộ khóa, dữ liệu

## Nhược điểm

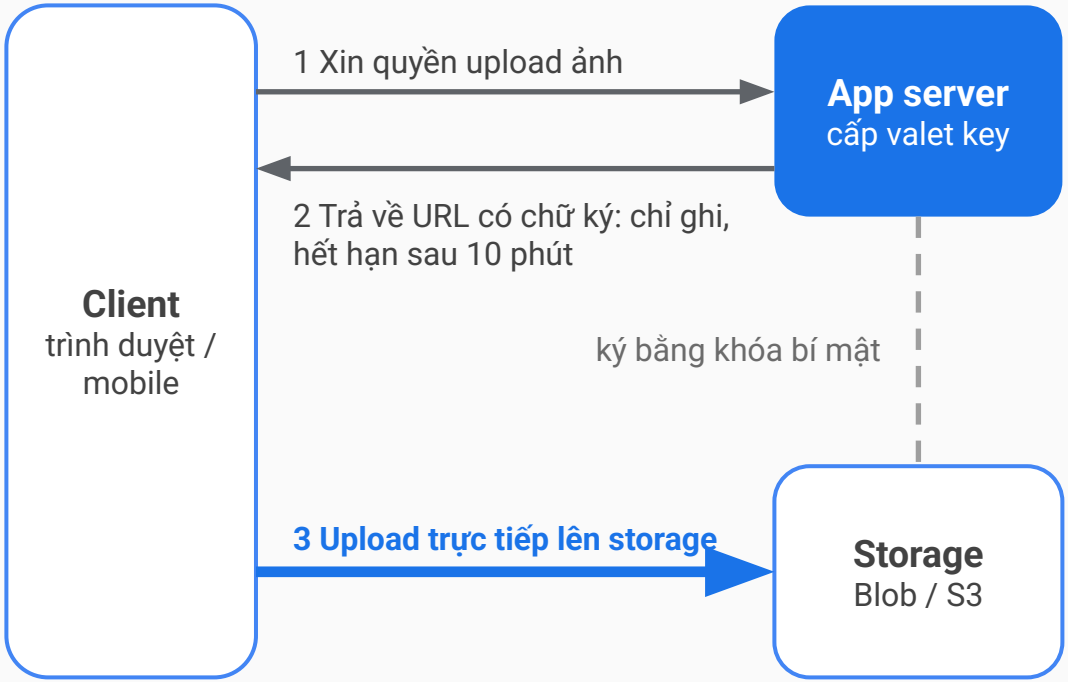
- Tăng độ trễ và phức tạp
- Dễ thành điểm nghẽn → chạy nhiều instance

# Valet Key



# Valet Key

Ý tưởng: cấp cho client một "chìa khóa có hạn" để truy cập thẳng tài nguyên, app server không phải làm trung gian.




## Ưu điểm

- App server không phải gánh băng thông file lớn
- Quyền tối thiểu: 1 tài nguyên, 1 thao tác, có hạn

## Lưu ý

- Hạn dùng ngắn, thu hồi được khi bị lộ
- Vẫn phải kiểm tra file sau khi upload

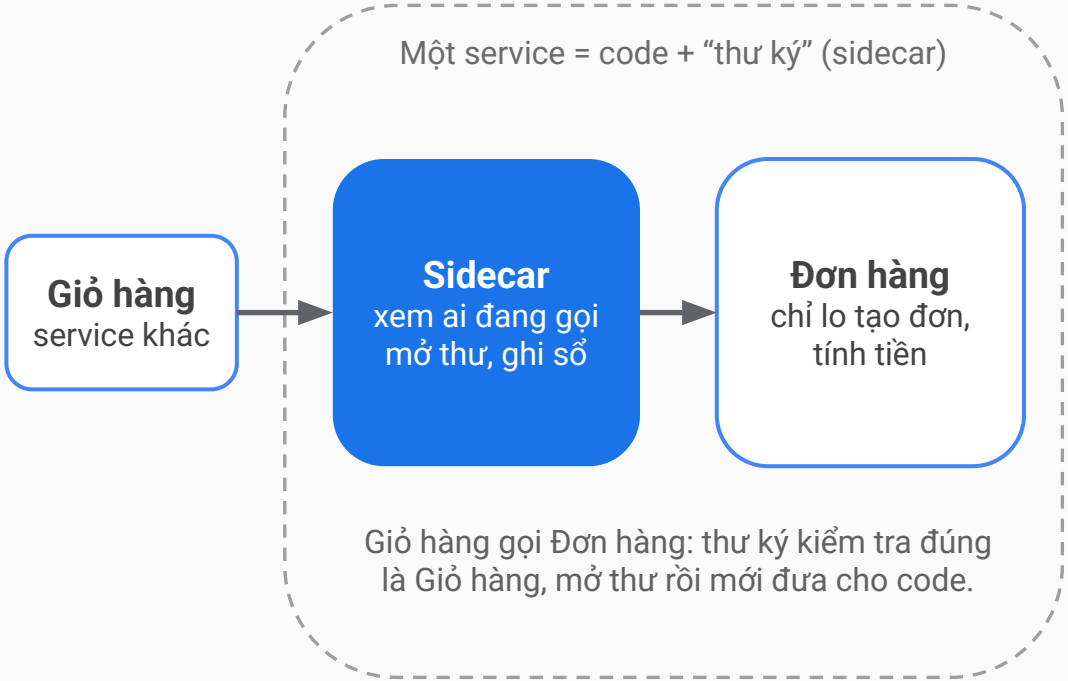
Ví dụ: Azure SAS token, AWS S3 pre-signed URL

# Sidecar



# Sidecar

Ý tưởng: mỗi service có một “thư ký riêng” đi kèm. Thư ký lo việc bảo mật, xem ai đang gọi, mở thư đã niêm phong, ghi sổ. Code chỉ lo việc chính.




## Ưu điểm

- Code chỉ lo nghiệp vụ, không tự lo bảo mật
- Đổi quy định bảo mật: thay thư ký, không sửa code

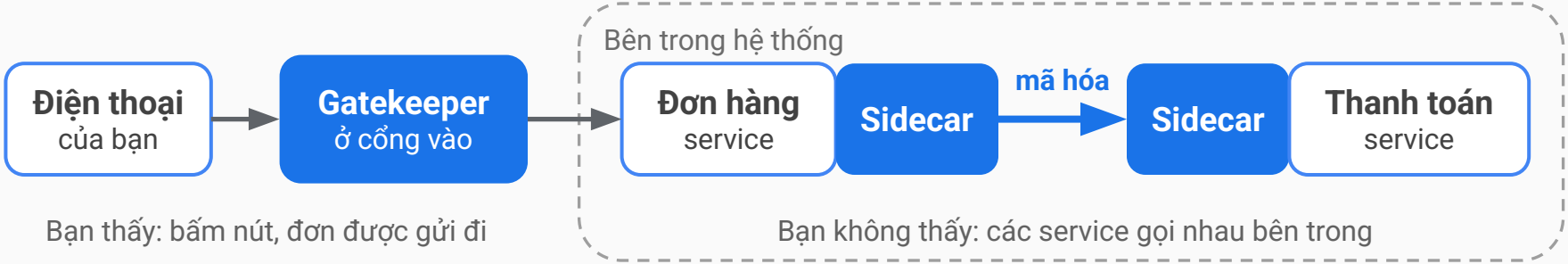
## Lưu ý

- Mọi lời gọi phải qua thư ký → chậm hơn chút
- Tốn thêm tài nguyên: service nào cũng kèm thư ký

Ví dụ: Lyft (app gọi xe như Grab) tạo ra Envoy làm “thư ký” cho hàng trăm service.

# Gatekeeper vs Sidecar

## Ví dụ: bạn bấm “Đặt hàng” trên app mua sắm




### Gatekeeper = bảo vệ ở cổng tòa nhà

Kiểm tra mọi người từ ngoài đi vào; cả tòa nhà dùng chung một chốt.

**Trong ví dụ:** kiểm tra bạn đã đăng nhập chưa, đơn có hợp lệ không, có phải bot đặt hàng loạt không.

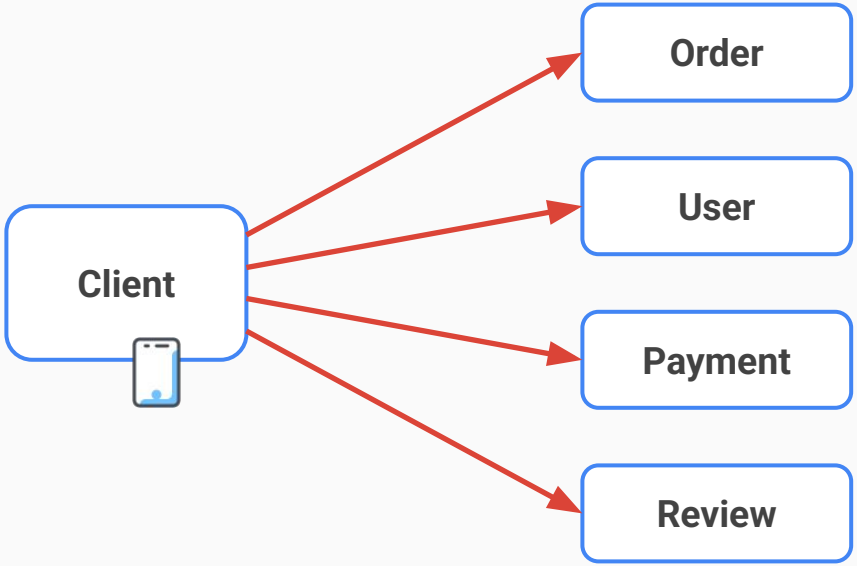
### Sidecar = thư ký riêng của từng phòng

Mỗi phòng có một người nhận, gửi giấy tờ thay sếp và kiểm tra người gửi; sếp chỉ lo việc chính.

**Trong ví dụ:** khi **Đơn hàng** gọi **Thanh toán**, sidecar mã hóa và xác nhận đúng là Đơn hàng đang gọi.

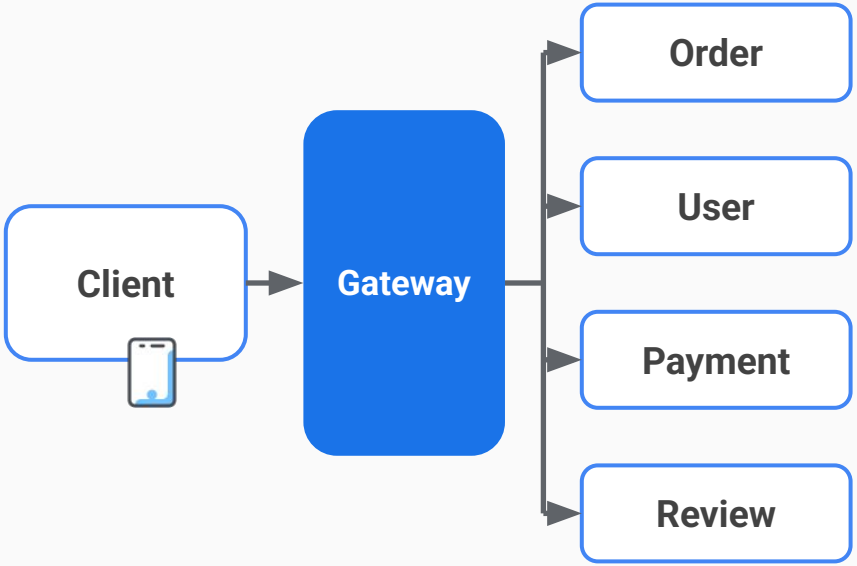
# API Gateway: vì sao cần?

## Không có gateway




Client biết địa chỉ nội bộ của mọi service; mỗi service tự lo TLS, xác thực, nhiều lượt gọi qua mạng di động chậm.

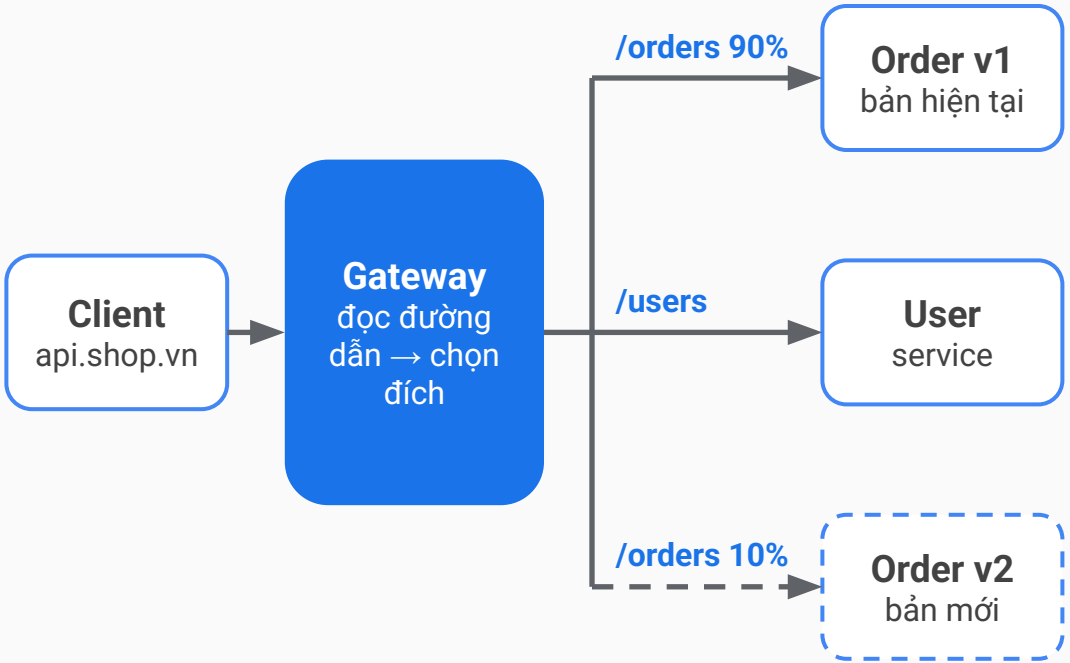
## Có API Gateway




Một cửa vào duy nhất: gateway lo định tuyến, gộp request và gánh hộ phần bảo mật dùng chung.

# Gateway Routing

Ý tưởng: client chỉ biết một địa chỉ; gateway đọc đường dẫn/header (lớp 7) rồi chuyển request tới đúng service.




## Ưu điểm

- Ẩn cấu trúc nội bộ, client chỉ biết 1 địa chỉ
- Cập nhật, đổi service mà client không biết

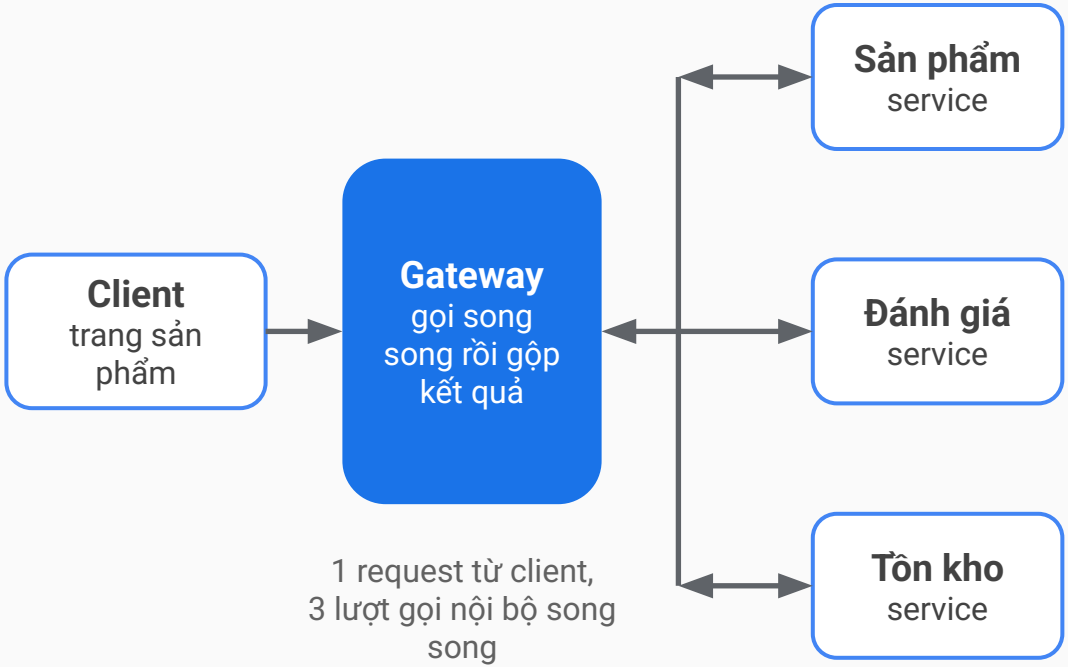
## Lưu ý

- Điểm nghẽn, điểm chết duy nhất → nhân bản
- Giữ luật định tuyến đơn giản

Ví dụ: Nginx, Kong, YARP, Azure Application Gateway

# Gateway Aggregation

Ý tưởng: client gửi một request, gateway gọi song song nhiều service rồi gộp kết quả trả về một lần.




## Ưu điểm

- Ít lượt gọi từ client, lợi với mobile, mạng chậm
- Client gọn; backend đổi không ảnh hưởng client

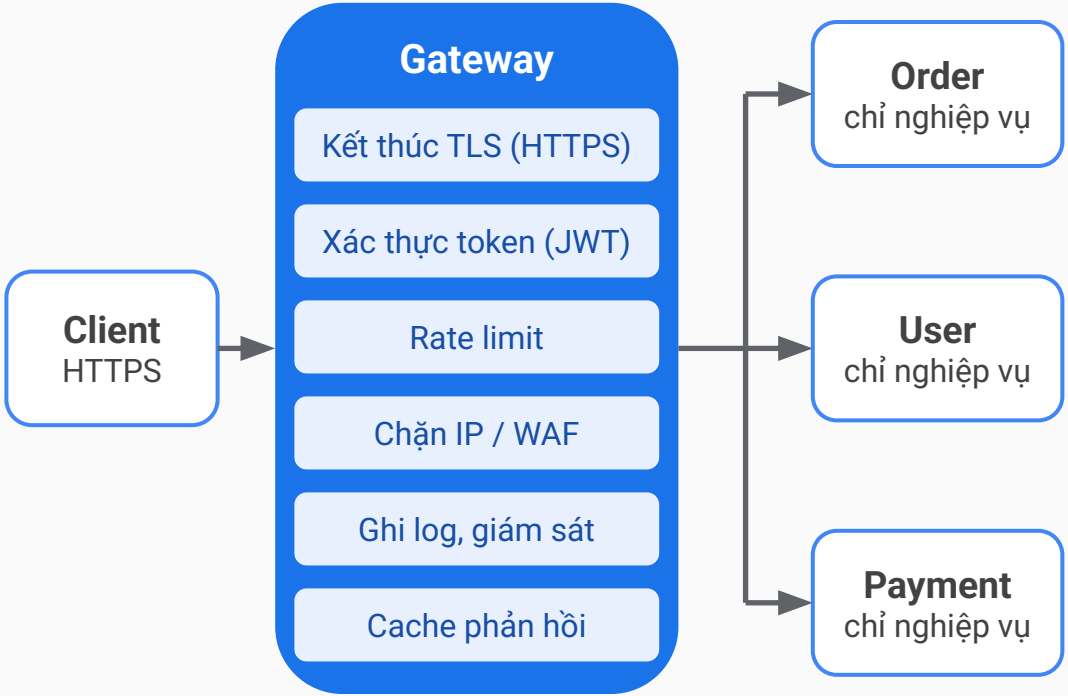
## Lưu ý

- Service lỗi: cần timeout, trả kết quả một phần
- Không đặt nghiệp vụ ở gateway

Ví dụ: Backend for Frontend (BFF), GraphQL gateway

# Gateway Offloading

Ý tưởng: chuyển các chức năng dùng chung (TLS, xác thực, giới hạn tần suất…) từ từng service lên gateway.




## Ưu điểm

- Viết và cấu hình một lần cho mọi service
- Đội nghiệp vụ không phải quản lý chứng chỉ, khóa

## Lưu ý

- Gateway không chứa logic nghiệp vụ
- Sau gateway vẫn cần đảm bảo tin cậy → kết hợp Sidecar

Ví dụ: Kong, Azure API Management, AWS API Gateway

# So sánh Gateway

| Mẫu | Giải quyết vấn đề gì | Quan hệ |
|---|---|---|
| **Gateway Routing** | Một địa chỉ, định tuyến tới nhiều service | 1 request → 1 service (chọn trong n) |
| **Gateway Aggregation** | Gộp nhiều lượt gọi thành một | 1 request → n service |
| **Gateway Offloading** | Gánh hộ TLS, xác thực, giới hạn tần suất | n service → 1 gateway làm hộ |

**Routing = chỉ đường**
**Aggregation = gom lại**
**Offloading = làm hộ**

# Trong thực tế

**Thực tế các mẫu đi cùng nhau:**
Gateway ở biên (routing + offloading)
Sidecar giữa các service
Valet Key cho file lớn.

# Thực hành: WAF trên Nginx (1/2)

Dựng một “người gác cổng” thật: Nginx + ModSecurity + bộ luật OWASP CRS đứng trước OWASP Juice Shop (web bán hàng cố ý có lỗ hổng), rồi thử tấn công nó.

```
# docker-compose.yml
services:
  web:
    image: bkimminich/juice-shop
    ports: ["3000:3000"]
  waf:
    image: owasp/modsecurity-crs:nginx
    ports: ["8080:8080"]
    environment:
      BACKEND: http://web:3000
      MODSEC_RULE_ENGINE: "On"
      BLOCKING_PARANOIA: "1"
```

## Giải thích

- BACKEND: WAF chuyển request hợp lệ tới đâu
- MODSEC_RULE_ENGINE: On = chặn, DetectionOnly = chỉ ghi log
- BLOCKING_PARANOIA: 1–4, càng cao càng dễ chặn nhầm
- Cổng 3000: vào thẳng web, không qua WAF
- Chạy: docker compose up -d

# Thực hành: WAF trên Nginx (2/2)

| Thử | Làm gì | Kết quả mong đợi |
|---|---|---|
| **Không có WAF** | Mở localhost:3000 → Login<br>Email: `' OR 1=1--` Mật khẩu: bất kỳ | **Vào được tài khoản admin!** |
| **Có WAF** | Mở localhost:8080, đăng nhập y hệt | **403 Forbidden, bị chặn** |
| **Đăng nhập đúng** | Qua :8080 với admin@juice-sh.op / admin123 | Vào bình thường |
| **Xem lý do** | `docker compose logs waf` | Thấy luật 942100 (SQL injection) |

# Thực hành: BTVN

**Xây dựng hệ thống gồm 2 service (User, Order) và 1 API Gateway (VD: nginx).**

- **Định tuyến /users và /orders tới đúng service (Routing)**
- **Kiểm tra JWT, giới hạn 10 request/giây tại gateway (Offloading)**
- **Endpoint /dashboard gộp thông tin user và đơn hàng của user đó (Aggregation)**
- **Upload ảnh đại diện lên MinIO bằng pre-signed URL hết hạn sau 5 phút (Valet Key)**
- **Đo độ trễ khi đi qua gateway so với gọi thẳng service**